PRACTICAL: 01

Aim: Explore Google Colab Platform.

Introduction to Google Colab:

Google Colab is a free, cloud-based platform for writing and executing Python code, particularly useful for data analysis, machine learning, and deep learning projects.

Exclusive Features of Google Colab:

- Free access to GPUs and TPUs
- Jupyter Notebook environment
- Real-time collaboration
- Integration with Google Drive
- Pre-installed popular Python libraries

Source Codes in Google Colab:

Python Code:

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left)

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

```
 else:
            right = mid - 1
    return -1
if __name__ == "__main__":
    sorted_array = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    target_value = 7

    result = binary_search(sorted_array, target_value)

    if result != -1:
        print(f"Element found at index: {result}")
    else:
        print("Element not found in the array.")
```

Output:

```
Element found at index: 6
```

Conclusion:

The provided Python code demonstrates the Selection Sort algorithm, a fundamental sorting technique known for its simplicity and ease of implementation.

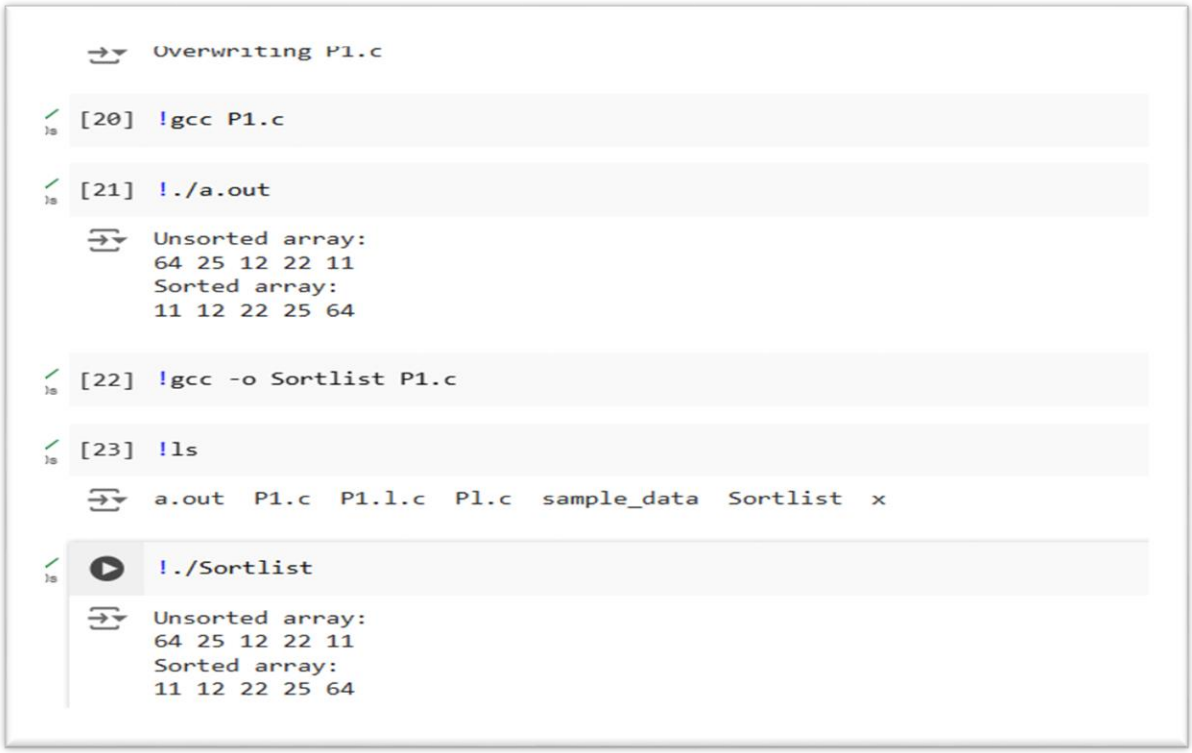
Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

C Code:

```
%%writefile Pl.c
#include "stdio.h"
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        // Swap the found minimum element with the first element
        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);
```

```
 printf("Unsorted array: \n");
    printArray(arr, n);
    selectionSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0;
}
```

Compilation and Execution Steps with Output:



Conclusion:

The provided C program demonstrates the Selection Sort algorithm, a straightforward and intuitive sorting technique.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

PRACTICAL: 02

Aim: Execute 20 Linux commands on google collab.

List of commands

1. date
   The date command displays the current date and time in the system's default format. It can also be used to format the output in various ways or set the system date and time.

```
[5]  !date

Wed Jan  8 06:01:11 AM UTC 2025
```

2. ls
   Lists the files and directories in the current working directory. Useful for checking the contents of a folder.

```
[6]  !ls

sample_data
```

3. cd
   Changes the current directory to the specified path. Use this to navigate through directories.

```
[8]  !cd
```

4. pwd
   Prints the current working directory path. Helps you understand where you are in the filesystem.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

```
[9]  !pwd
```

5. mkdir
   Creates a new directory with the specified name. Useful for organizing files into separate folders.

```
[11]  !mkdir shawul_directory
```

6. rmdir
   Removes an empty directory. Use this to clean up directories that are no longer needed.

```
[12]  !rmdir shawul_directory
```

7. cp
   Copies a file from the source to the destination. This is useful for backing up files or duplicating them.

```
[14]  !cp -r Eclipse IDE shawul_directory
```

8. mv
   Moves or renames a file. This command is handy for organizing files or changing their names.

```
[2]  !mv shawul_directory shawul_directory1
```

9. touch
   The command is used to create new, empty files quickly.

```
[4]  !touch hpc.txt
```

10. rm

    Deletes a specified file. Be cautious with this command, as it permanently removes files.

```
[5]  !rm hpc.txt
```

11. echo

    The echo command outputs the specified text or variables to the terminal. It is commonly used to display messages, create files with content, or print the values of environment variables.

```
[6]  !echo "This is a sample text.High perfomance computing." > hpc.tx
```

12. cat

    The cat command concatenates and displays the contents of one or more files in the terminal. It is commonly used to quickly view text files, combine multiple files, or create new files by redirecting output.

```
[7]  !cat hpc.txt

     This is a sample text.High perfomance computing
```

13. head

    Shows the first few lines of a file (default is 10). This is useful for previewing file contents without opening the entire file.

```
[8]  !head hpc.txt
```

14. tail

    Displays the last few lines of a file (default is 10). Useful for checking the end of log files or large text files.

```
[9]  !tail hpc.txt
```

15. df -h

Displays disk space usage for all mounted filesystems in a human-readable format. Useful for monitoring available disk space.

```
[10] !df -h
```

```
Filesystem      Size  Used Avail Use% Mounted on
overlay         108G   33G   76G  31% /
tmpfs            64M     0   64M   0% /dev
shm             5.8G     0  5.8G   0% /dev/shm
/dev/root       2.0G  1.2G  820M  59% /usr/sbin/docker-init
tmpfs           6.4G  528K  6.4G   1% /var/colab
/dev/sda1        50G   34G   17G  68% /etc/hosts
tmpfs           6.4G     0  6.4G   0% /proc/acpi
tmpfs           6.4G     0  6.4G   0% /proc/scsi
tmpfs           6.4G     0  6.4G   0% /sys/firmware
```

16. free -h

Shows memory usage, including total, used, and free memory. This helps you understand the system's memory status.

```
[11] !free -h
```

```
               total        used        free      shared  buff/cache   available
Mem:            12Gi       814Mi       8.6Gi       2.0Mi       3.2Gi        11Gi
Swap:            0B          0B          0B
```

17. ps aux

Displays a snapshot of the current processes running on the system. It provides information such as process IDs (PIDs), terminal associated, CPU usage, and memory usage, helping you monitor system activity.

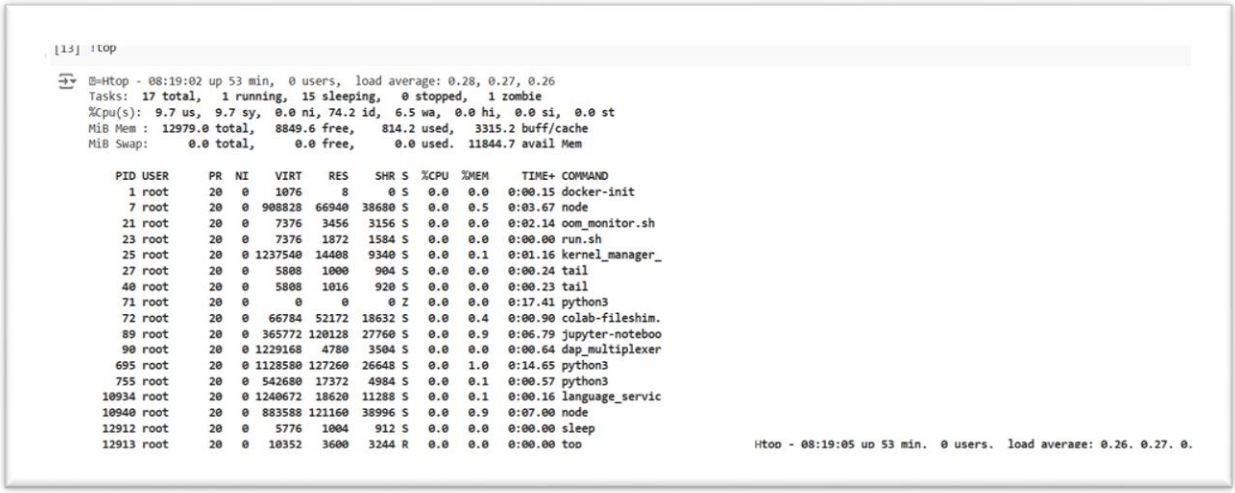
```
[12] !ps aux
```

```
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0   1076     8 ?        Ss   07:25   0:00 /sbin/docker-init -- /datalab/run
root           7  0.1  0.5 908828 66976 ?        Sl   07:25   0:03 /tools/node/bin/node /datalab/web
root          21  0.0  0.0   7376  3456 ?        S    07:25   0:02 /bin/bash -e /usr/local/colab/bin
root          23  0.0  0.0   7376  1872 ?        S    07:25   0:00 /bin/bash -e /datalab/run.sh
root          25  0.0  0.1 1237540 14668 ?       Sl   07:25   0:01 /usr/colab/bin/kernel_manager_pro
root          27  0.0  0.0   5808  1000 ?        Ss   07:25   0:00 tail -n +0 -F /root/.config/Googl
root          40  0.0  0.0   5808  1016 ?        Ss   07:25   0:00 tail -n +0 -F /root/.config/Googl
root          71  0.5  0.0      0     0 ?        Z    07:25   0:17 [python3] <defunct>
root          72  0.0  0.3  66784 52172 ?        S    07:25   0:00 python3 /usr/local/bin/colab-file
root          89  0.2  0.9 365772 120128 ?       Sl   07:25   0:06 /usr/bin/python3 /usr/local/bin/j
root          90  0.0  0.0 1229168 4780 ?        Sl   07:25   0:00 /usr/local/bin/dap_multiplexer --
root         695  0.4  0.9 1128580 126996 ?      Ssl  07:28   0:14 /usr/bin/python3 -m colab_kernel_
root         755  0.0  0.1 542680 17372 ?        Sl   07:28   0:00 /usr/bin/python3 /usr/local/lib/p
root       10934  0.0  0.1 1240672 18620 ?       Sl   08:10   0:00 /usr/colab/bin/language_service -
root       10940  1.7  0.9 883076 119840 ?       Sl   08:10   0:06 node /datalab/web/pyright/pyright
root       12483  0.0  0.0   5776  1052 ?        S    08:17   0:00 sleep 1
root       12484  0.0  0.0  10076  1628 ?        R    08:17   0:00 ps aux
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

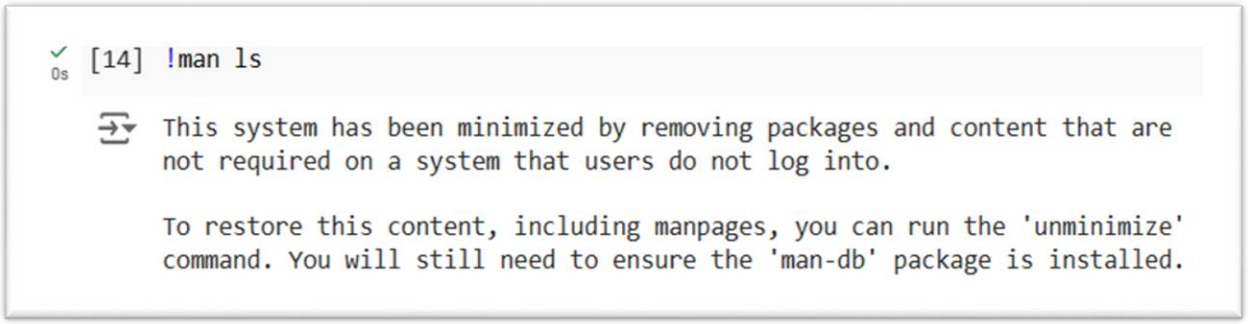
18. top

Displays a dynamic, real-time view of system processes, including CPU and memory usage. It helps monitor system performance and resource consumption.



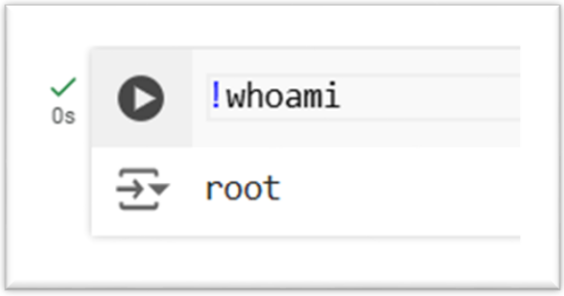
19. man ls

Opens the manual page for the ls command, providing detailed information about its usage, options, and examples. This is useful for understanding how to use commands effectively.



20. whoami

Prints the username of the currently logged-in user. This command is helpful for confirming your identity in a multi-user environment.



Conclusion:

Running Linux commands in environments like Google Colab is a practical skill in HPC environments, where Linux is often the underlying OS. These commands help in tasks like managing files, checking system resources (disk, memory, CPU), installing dependencies, and monitoring system status—all of which are essential in managing HPC resources efficiently. Through such commands, users can manage computational workflows, debug system performance issues, and perform system optimizations, which are core activities in HPC management.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

PRACTICAL: 03

Aim: Implementation of Quick Sort.

What is Divide & Conquer Strategy:

- Divide: The problem is divided into smaller, more manageable sub-problems of the same type.
- Conquer: Each sub-problem is solved independently, often recursively.
- Combine: The solutions of the sub-problems are combined to form the solution to the original problem.

Advantages of Divide & Conquer:

- **Efficiency:** It reduces the problem size significantly, making algorithms like Merge Sort and Quick Sort faster.
- **Parallelism:** Sub-problems can often be solved independently, enabling parallel processing for better performance.
- **Simplified Complexity:** It breaks complex problems into smaller, easier-to-handle pieces.
- **Reusable Sub-solutions:** Solutions to sub-problems can often be reused, as in Dynamic Programming.
- **Versatility:** It is widely applicable to a variety of problems, including sorting, searching, and optimization.

Quick sort algorithm steps:

- **Choose a Pivot:** Select a pivot element from the array. Common choices include the first element, the last element, the middle element, or a random element.
- **Partition the Array:** Rearrange the elements so that:
  - All elements smaller than the pivot are placed to its left.
  - All elements greater than the pivot are placed to its right. This forms two sub-arrays.
- **Recursively Sort Sub-arrays:** Apply the Quick Sort algorithm to the left and right sub-arrays created in the partition step.
- **Combine:** The base case is reached when sub-arrays have one or zero elements, which are already sorted. Combine the sorted sub-arrays to produce the final sorted array.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Quick sort code:

```
def quick_sort(arr):
    if len(arr) < 2:
        return arr
    else:
        pivot = arr[0]
        less = [i for i in arr[1:] if i <= pivot]
        greater = [i for i in arr[1:] if i > pivot]
        return quick_sort(less) + [pivot] + quick_sort(greater)
# Example usage
my_list = [5, 1, 9, 3, 7, 4, 6, 2, 8]
sorted_list = quick_sort(my_list)
sorted_list
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Conclusion:

The implementation of Quick Sort demonstrates its efficiency and versatility in sorting data. It utilizes the divide-and-conquer strategy to break the problem into smaller sub-problems, recursively sorting them. By choosing an appropriate pivot and partitioning effectively, Quick Sort achieves an average time complexity of O(n log n).

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

PRACTICAL: 04

Aim: Demonstrate MPI functions through a simple program.

What is MPI?

MPI (Message Passing Interface) is a standardized and portable message-passing system used to program parallel computers. It enables multiple processes to communicate with each other, typically in a distributed computing environment. MPI is widely used in high-performance computing (HPC) to run applications across multiple nodes in a cluster.

Key Features of MPI:

- Scalability: Works on small to supercomputing scales.
- Efficiency: Optimized for high-speed communication between processes.
- Portability: Runs on various hardware and operating systems.

Basic MPI functions:

1. **MPI_Init(&argc, &argv)** – Initializes the MPI environment.
2. **MPI_Finalize()** – Terminates the MPI environment.
3. **MPI_Comm_size(MPI_COMM_WORLD, &size)** – Gets the total number of processes.
4. **MPI_Comm_rank(MPI_COMM_WORLD, &rank)** – Gets the rank (ID) of the process.
5. **MPI_Send(buffer, count, datatype, dest, tag, comm)** – Sends data to a process.
6. **MPI_Recv(buffer, count, datatype, source, tag, comm, &status)** – Receives data from a process.
7. **MPI_Bcast(buffer, count, datatype, root, comm)** – Broadcasts data from one process to all.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Simple MPI C Program:

```
#include <stdio.h>
   #include <mpi.h>

   int main(int argc, char** argv){
    int process_Rank, size_Of_Cluster;

      MPI_Init(&argc, &argv);
      MPI_Comm_size(MPI_COMM_WORLD, &size_Of_Cluster);
      MPI_Comm_rank(MPI_COMM_WORLD, &process_Rank);
printf("Hello World from process %d of %d\n", process_Rank, size_Of_Cluster);

      MPI_Finalize();
     return 0;
 }
```

Compilation and Execution Steps:

1. Create a host file and store the content

   ```
   labex:mpi1/ $ vi my-hostfile
   ```

   Write following inside it: localhost slots=10 and save

   ```
   labex:mpi1/ $ cat my-hostfile
   localhost slots=10
   ```

2. Write a C program
3. Compile it

   mpicc p1.c -o x

   ```
   labex:project/ $ mpicc p1.c -o x
   ```

4. Run it

   mpirun -np 7 --hostfile my-hostfile x

   ```
   labex:project/ $ mpirun -np 7 --hostfile my-hostfile x
   ```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Output:

```
labex:project/ $ mpirun -np 7 --hostfile my-hostfile x
Hello World from process 3 of 7
Hello World from process 1 of 7
Hello World from process 4 of 7
Hello World from process 6 of 7
Hello World from process 5 of 7
Hello World from process 2 of 7
Hello World from process 0 of 7
labex:project/ $
```

Conclusion:

This MPI program initializes a parallel environment, retrieves the total number of processes and the rank of each process, then prints a "Hello World" message from each process before finalizing MPI.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

PRACTICAL: 05

Aim: Write a program to check task distribution using Gprof.

what is gprof:

- A profiler is a tool that can help you analyze an application's performance by providing visual representations of CPU usage and execution times. This can help you quickly identify and fix issues.
- Also, Gprof is a profiling program which collects and arranges statistics on your programs. Basically, it looks into each of your functions and inserts code at the head and tail of each one to collect timing information (actually, I don't believe it checks each time the function is run, but rather collects statistically significant samples). Then, when you run your program normally (that means with any std and file i/o you would normally have), it creates "gmon.out"... raw data which the gprof program turns into profiling statistics (which tell you all sorts of neat stuff).

Applications of a profiler:

1) A profiler can help you start and stop a profiling session for a local application. You can use the profiler to refresh results, save data, and invoke garbage collection.
2) Identify Bottlenecks: Profilers help locate slow parts of the code, allowing developers to focus on optimizing those areas.
3) Resource Utilization: They provide insights into CPU, memory, and I/O usage, helping to optimize resource allocation.

g prof steps on a program:

- Compile and link the program with the -pg flag.
- Run the program to generate a profile data file.
- Run gprof to interpret the profile data file.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Code:

```
%%writefile prac_5a.c


#include <stdio.h>
int add(int a, int b) {return a + b;
}
int sub(int a, int b) {return a - b;
}
int main() {
   int num1, num2, choice, result;
printf("Enter two integers: ");
scanf("%d %d", &num1, &num2);
printf("Choose an operation:\n");
printf("1. Addition\n");
printf("2. Subtraction\n");
printf("Enter uourchoice (1 or 2):");
scanf("%d", &choice);
if (choice == 1) {result = add(num1, num2);
printf("The result of addition is: %d\n", result);
   } else if (choice == 2) {result = sub(num1, num2);
      printf("The result of subtraction is: %d\n", result);
   } else {
      printf("Invalid choice!\n");
   }
return 0;
}
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Output:

```
[2]  !gcc -Wall -pg prac_5a.c -o prac_5a
     !./prac_5a
     !ls
     !gprof prac_5a gmon.out > op.txt
     !cat op.txt

     Enter two integers: 12 12
     Choose an operation:
     1. Addition
     2. Subtraction
     Enter uourchoice (1 or 2):1
     The result of addition is: 24
     gmon.out  prac_5a  prac_5a.c  sample_data
     Flat profile:

     Each sample counts as 0.01 seconds.
      no time accumulated

       %   cumulative   self              self     total
      time   seconds   seconds    calls  Ts/call  Ts/call  name
       0.00      0.00     0.00        1     0.00     0.00  add
```

Code:

```
%%writefile prac_5a.c

#include <stdio.h>

#include <stdbool.h>

bool is_prime(int num) {

    if (num <= 1) return false;

    for (int i = 2; i * i <= num; i++) {

        if (num % i == 0) return false;

    }

    return true;

}
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

```
int main() {
    int n, count = 0, sum = 0, current_number = 2;
    printf("Enter the number of prime numbers to sum: ");
    scanf("%d", &n);
    while (count < n) {
        if (is_prime(current_number)) {
            sum += current_number;
            count++;
        }
        current_number++;
    }
    printf("The sum of the first %d prime numbers is: %d\n", n, sum);
    return 0;
}
```

Output:

```
[4]  !gcc -Wall -pg prac_5a.c -o prac_5a
     !./prac_5a
     !ls
     !gprof prac_5a gmon.out > op.txt
     !cat op.txt

     Enter the number of prime numbers to sum: 1000000000000000
     The sum of the first -1530494976 prime numbers is: 0
     gmon.out  op.txt  prac_5a  prac_5a.c  sample_data
     Flat profile:

     Each sample counts as 0.01 seconds.
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

PRACTICAL: 06

Aim: Write a simple CUDA program to print "Hello World!"

What is cuda programming?

CUDA (Compute Unified Device Architecture) is a parallel computing platform and application programming interface (API) model created by NVIDIA. It allows developers to use a CUDA-enabled graphics processing unit (GPU) for general-purpose processing, an approach known as GPGPU (General-Purpose computing on Graphics Processing Units). Here are some key aspects of CUDA programming.

GPU programming unit layers

1. **Grid**
   • **Description:** A grid is the highest-level organizational unit in CUDA programming. It represents the entire collection of thread blocks that execute a kernel (a function that runs on the GPU).
2. **Block**
   • **Description:** A block is a group of threads that execute the same kernel. Each block can contain a maximum number of threads, which is determined by the GPU architecture (typically up to 1024 threads per block).
3. **Warp**
   • **Description:** A warp is a group of threads that are executed simultaneously by the GPU's streaming multiprocessors (SMs). In NVIDIA GPUs, a warp typically consists of 32 threads.
4. **Thread**
   • **Description:** A thread is the smallest unit of execution in CUDA. Each thread executes a kernel and has its own local memory, registers, and execution context.

Steps of cuda program execution:

Step 1: Create the CUDA Source File

Step 2: Compile the CUDA Program

Step 3: Run the CUDA Program

Step 4: Observe the Output Step 5: Clean Up

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

List of cuda functions to be used:

1) **cudaMalloc():** Allocates memory on the device (GPU).
2) **cudaFree():** Frees memory that was previously allocated on the device.
3) **cudaMemcpy():** Copies data between host (CPU) and device (GPU) memory. It can be used for copying data to and from the GPU
4) **cudaDeviceSynchronize():** This function blocks the host (CPU) until all previously launched kernels on the device (GPU) have completed. It ensures that the CPU waits for the GPU to finish executing before proceeding, which is important for ensuring that all output from the GPU is printed before the program exits.

Simple cuda program to print hello world:

```
%%writefile p6_hpc.cu

#include<stdio.h>

#include<cuda.h>

#include<cuda_runtime_api.h>


__global__ void ns()

{

  printf("\n Hello World from thread %d of block %d",threadIdx.x,blockIdx.x);

}

 int  main(void)

{

  ns<<<6,2>>>(); // 1,1 refers to blocks and threads

  cudaDeviceSynchronize();

  return 0;

}
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Output:

Cuda program with multiple blocks and multiple blocks and threads:

```
%%writefile p6_hpc.cu
#include<stdio.h>
#include<cuda.h>
#include<cuda_runtime_api.h>

__global__ void ns()
{
  printf("\n Hello World from thread %d of block %d",threadIdx.x,blockIdx.x);
}
 int  main(void)
{
  ns<<<6,2>>>(); // 1,1 refers to blocks and threads
  cudaDeviceSynchronize();
  return 0;
}
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Output:

```
[8]  !nvcc -o x p6_hpc.cu
     !./x

Hello World from thread 0 of block 4
Hello World from thread 1 of block 4
Hello World from thread 0 of block 1
Hello World from thread 1 of block 1
Hello World from thread 0 of block 3
Hello World from thread 1 of block 3
Hello World from thread 0 of block 0
Hello World from thread 1 of block 0
Hello World from thread 0 of block 5
Hello World from thread 1 of block 5
Hello World from thread 0 of block 2
Hello World from thread 1 of block 2
```

Conclusion:

Together, these programs serve as an introduction to CUDA programming, illustrating how to define and launch kernels, utilize multiple threads and blocks, and synchronize execution. They provide a foundation for understanding more complex GPU programming tasks and highlight the power of parallel processing in modern computing.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

PRACTICAL: 07

Aim: Write a simple CUDA program to add two numbers.

What is CUDA?

CUDA is a parallel computing platform and programming model created by NVIDIA. With more than 20 million downloads to date, CUDA helps developers speed up their applications by harnessing the power of GPU accelerators.

Basic CUDA functions:

In high performance computing, fundamental CUDA functions include: "cudaMalloc" for allocating memory on the GPU, "cudaMemcpy" for transferring data between host (CPU) and device (GPU) memory, "cudaLaunchKernel" to launch a CUDA kernel (parallel computation), and "cudaThreadSynchronize" to ensure all GPU threads finish before proceeding on the host; these functions are crucial for efficiently utilizing the parallel processing power of NVIDIA GPUs for computationally intensive tasks.

CUDA program to add two numbers:

```
%%writefile P7.cu

#include<stdio.h>

#include<cuda.h>

#include<cuda_runtime_api.h>


__global__ void AddintsCUDA(int *a,int *b)

{

  printf("\n Hello Shawul from GPU..!");

  *a=*a+*b;

}
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

```
int main()
{
   int a=5,b=9;
   int*d_a,*d_b;

   cudaMalloc((void**)&d_a,sizeof(int));
   cudaMalloc((void**)&d_b,sizeof(int));

   cudaMemcpy(d_a,&a,sizeof(int),cudaMemcpyHostToDevice);
   cudaMemcpy(d_b,&b,sizeof(int),cudaMemcpyHostToDevice);

   AddintsCUDA<<<1,1>>>(d_a,d_b);

   cudaMemcpy(&a,d_a,sizeof(int),cudaMemcpyDeviceToHost);

   printf("\n The answer is : %d",a);

   cudaFree(d_a);
   cudaFree(d_b);

   return 0;
}
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Compilation and Execution:

```
[ ]  !nvcc -arch=sm_75 P7.cu -o x
```

Output:

```
[ ]  !./x

     Hello Shawul from GPU..!
     The answer is : 14
```

Conclusion:

This program demonstrates the basic structure of a CUDA program, including memory management, kernel execution, and data transfer between the host and device. While this example is trivial, it lays the foundation for more complex GPU-accelerated computations. CUDA enables efficient parallel processing by leveraging the power of GPUs, making it ideal for computationally intensive tasks.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

PRACTICAL: 08

Aim: Write a CUDA program to add two arrays.

What is CUDA?

CUDA is a parallel computing platform and programming model created by NVIDIA. With more than 20 million downloads to date, CUDA helps developers speed up their applications by harnessing the power of GPU accelerators.

What is blockIdx.x,blockIdx.y,threadIdx.x,,threadIdx.y values

In CUDA:

- blockIdx.x, blockIdx.y → Block's index in the grid.
- threadIdx.x, threadIdx.y → Thread's index in the block.

These helps uniquely identify each thread in parallel computing.

Program to add two arrays:

```
%%writefile P8.cu

#include <stdio.h>

#include <cuda.h>


__global__ void arradd(int *x, int *y, int *z)

{

   int id = blockIdx.x;

   z[id] = x[id] + y[id];

}


int main()

{
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

```
int a[6] = {1, 2, 3, 4, 5, 6};
int b[6] = {10, 20, 30, 40, 50, 60};
int c[6];
int *d, *e, *f;
int i;
cudaMalloc((void**)&d, 6 * sizeof(int));
cudaMalloc((void**)&e, 6 * sizeof(int));
cudaMalloc((void**)&f, 6 * sizeof(int));

cudaMemcpy(d, a, 6 * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(e, b, 6 * sizeof(int), cudaMemcpyHostToDevice);

arradd<<<6, 1>>>(d, e, f);

cudaMemcpy(c, f, 6 * sizeof(int), cudaMemcpyDeviceToHost);

printf("Array A:\n");
for (i = 0; i < 6; i++)
{
   printf("%d ", a[i]);
}
printf("\n\n");

printf("Array B:\n");
for (i = 0; i < 6; i++)
{
```

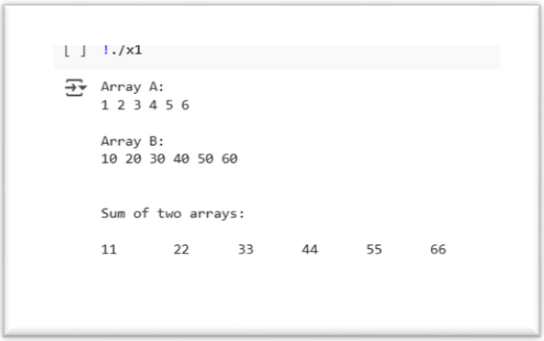
```
 printf("%d ", b[i]);
    }
    printf("\n\n");

    printf("\nSum of two arrays:\n\n");
    for (i = 0; i < 6; i++)

    {
        printf("%d\t ", c[i]);
    }
    printf("\n");

    cudaFree(d);
    cudaFree(e);
    cudaFree(f);

    return 0;
}
```

Compilation and Execution:

```
!nvcc -arch=sm_75 P8.cu -o x1
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Output:



Conclusion:

This CUDA program adds two arrays element-wise using GPU parallel processing. Each thread handles one element, utilizing blockIdx.x for indexing. The result is copied back to the CPU and displayed.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

PRACTICAL: 09

Aim: Analyze the code using Nvidia-Profilers.

What is a profiler?

A profiler is a tool or software that helps developers analyze the performance of their applications by measuring various aspects of their execution. Profilers provide insights into how an application uses resources such as CPU, memory, and I/O, allowing developers to identify bottlenecks, optimize performance, and improve overall efficiency.

Introduction to nvprof:

nvprof is a command-line profiling tool from NVIDIA designed for analyzing the performance of CUDA applications. It collects detailed profiling data, allowing developers to understand where their code spends time and how resources are utilized, ultimately aiding in performance optimization.

Features of nvprof:

- Performance Analysis: nvprof provides insights into the execution time of CUDA kernels and memory operations, helping identify bottlenecks in the application.
- Resource Utilization: It tracks how effectively the GPU resources are being used, including memory transfers and kernel execution.
- Command-Line Interface: Being a command-line tool, nvprof can be easily integrated into scripts and automated workflows.
- Support for Multiple Languages: It can profile both compiled CUDA code and Python libraries that utilize CUDA, making it versatile for different development environments.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Nvprof steps on CUDA Program:

Step 1: Prepare Your CUDA Program:

Ensure that your CUDA program is correctly set up and compiled. You should have a working CUDA application that you want to profile. For example, let's say you have a simple CUDA program named “p6_hpc.cu”.

```
[1] %%writefile p6_hpc.cu
    #include<stdio.h>
    #include<cuda.h>
    #include<cuda_runtime_api.h>

    __global__ void ns()
    {
      printf("\n Hello World from thread %d of block %d",threadIdx.x,blockIdx.x);
    }
     int  main(void)
    {
      ns<<<6,2>>>(); // 1,1 refers to blocks and threads
      cudaDeviceSynchronize();
      return 0;
    }
```

Step 2: Compile Your CUDA Program:

Compile your CUDA program using nvcc, the NVIDIA CUDA compiler.

```
[2] !nvcc -o x p6_hpc.cu
    !./x

 Hello World from thread 0 of block 4
 Hello World from thread 1 of block 4
 Hello World from thread 0 of block 1
 Hello World from thread 1 of block 1
 Hello World from thread 0 of block 3
 Hello World from thread 1 of block 3
 Hello World from thread 0 of block 0
 Hello World from thread 1 of block 0
 Hello World from thread 0 of block 5
 Hello World from thread 1 of block 5
 Hello World from thread 0 of block 2
 Hello World from thread 1 of block 2
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Step 3: Run Your Program with nvprof:

To profile your CUDA application, run it with **nvprof** from the command line. The basic command is:

```
[3] !nvprof ./x

==2662== NVPROF is profiling process 2662, command: ./x
 Hello World from thread 0 of block 4
 Hello World from thread 1 of block 4
 Hello World from thread 0 of block 1
 Hello World from thread 1 of block 1
 Hello World from thread 0 of block 0
 Hello World from thread 1 of block 0
 Hello World from thread 0 of block 5
 Hello World from thread 1 of block 5
 Hello World from thread 0 of block 3
 Hello World from thread 1 of block 3
 Hello World from thread 0 of block 2
 Hello World from thread 1 of block 2==2662== Profiling application: ./x
==2662== Profiling result:
            Type  Time(%)      Time     Calls       Avg       Min       Max  Name
 GPU activities:  100.00%  101.73us         1  101.73us  101.73us  101.73us  ns(void)
      API calls:   99.70%  137.57ms         1  137.57ms  137.57ms  137.57ms  cudaLaunchKernel
                    0.14%  194.81us       114  1.7080us     210ns  70.644us  cuDeviceGetAttribute
                    0.14%  194.57us         1  194.57us  194.57us  194.57us  cudaDeviceSynchronize
                    0.01%  13.086us         1  13.086us  13.086us  13.086us  cuDeviceGetName
                    0.01%  7.3260us         1  7.3260us  7.3260us  7.3260us  cuDeviceGetPCIBusId
                    0.00%  6.1490us         1  6.1490us  6.1490us  6.1490us  cuDeviceTotalMem
                    0.00%  1.9970us         3     665ns     334ns  1.1750us  cuDeviceGetCount
                    0.00%  1.0930us         2     546ns     370ns     723ns  cuDeviceGet
                    0.00%     999ns         1     999ns     999ns     999ns  cuModuleGetLoadingMode
                    0.00%     544ns         1     544ns     544ns     544ns  cuDeviceGetUuid
```

Step 4: Analyze the Output:

After running the command, **nvprof** will output profiling information directly to the console. This includes:

- The execution time of each CUDA kernel.
- Memory transfer times between the host and device.

Conclusion:

Using nvprof to profile your CUDA applications is a straightforward process that can provide valuable insights into performance. By following these steps, you can effectively analyze and optimize your CUDA programs for better performance on NVIDIA GPUs.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

PRACTICAL: 10

Aim: Demonstration of OpenMP and p thread functions.

What is pthtead?

pthread refers to the **POSIX Threads** (Portable Operating System Interface for Unix) library, which provides a standardized API for creating and managing threads in a multi-threaded programming environment. It is widely used in Unix-like operating systems, including Linux and macOS, to develop concurrent and parallel programs.

Functions of pthread library

1. ***pthread_create:*** used to create a new thread
2. ***pthread_exit:*** used to terminate a thread
3. ***pthread_join:*** used to wait for the termination of a thread.
4. ***pthread_self:*** used to get the thread id of the current thread.
5. ***pthread_equal:*** compares whether two threads are the same or not. If the two threads are equal, the function returns a non-zero value otherwise zero.
6. ***pthread_detach:*** used to detach a thread. A detached thread does not require a thread to join on terminating. The resources of the thread are automatically released after terminating if the thread is detached.

Simple pthread program:

```
%%writefile pthread.c

#include <pthread.h>

#include <stdio.h>

#include <stdlib.h>

#define NUM_THREADS 5

void *PrintHello(void *threadid) {

   int tid = *((int *)threadid);
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

```
printf("Hello World! It's me, thread #%d!\n", tid);
    free(threadid);
    pthread_exit(NULL);
}
int main(int argc, char *argv[]) {
    pthread_t threads[NUM_THREADS];
    int rc;
    int t;

    for (t = 0; t < NUM_THREADS; t++) {
        printf("In main: creating thread %d\n", t);

        int *thread_id = malloc(sizeof(int));
        *thread_id = t;
        rc = pthread_create(&threads[t], NULL, PrintHello, (void *)thread_id);
        if (rc) {
            printf("ERROR; return code from pthread_create() is %d\n", rc);
            free(thread_id);
        }
    }
    for (t = 0; t < NUM_THREADS; t++) {
        pthread_join(threads[t], NULL);
    }
    pthread_exit(NULL);
}
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Compilation and Execution Steps:

Step-1:

```
[2]  !gcc -o pthread pthread.c -lpthread
```

Step-2:

```
[3]  !./pthread
```

Output:

```
[3]  !./pthread

In main: creating thread 0
In main: creating thread 1
In main: creating thread 2
In main: creating thread 3
In main: creating thread 4
Hello World! It's me, thread #2!
Hello World! It's me, thread #1!
Hello World! It's me, thread #4!
Hello World! It's me, thread #0!
Hello World! It's me, thread #3!
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Aim: Demonstration of OpenMP

What is OpenMP?

OpenMP (Open Multi-Processing) is a widely-used application programming interface (API) that supports multi-platform shared memory multiprocessing programming in C, C++, and Fortran. It provides a straightforward and flexible way to develop parallel applications by enabling developers to use multi-threading to execute code simultaneously on multiple processor cores.

Functions of OpenMP

1. Thread Management Functions

These functions control thread behavior and provide information about the threads.

- **omp_get_thread_num()**: Returns the ID of the current thread (from 0 to N-1, where N is the total number of threads).
- **omp_get_num_threads()**: Returns the total number of threads currently in the team.
- **omp_get_max_threads()**: Returns the maximum number of threads available.
- **omp_set_num_threads(int num_threads)**: Sets the number of threads for subsequent parallel regions.
- **omp_get_num_procs()**: Returns the number of processors available to the program.

2. Parallel Region Control

These functions help determine whether the program is running in a parallel region and manage nested parallelism.

- **omp_in_parallel()**: Returns a non-zero value if the program is currently in a parallel region.
- **omp_set_nested(int)**: Enables or disables nested parallelism (default is disabled).
- **omp_get_nested()**: Checks whether nested parallelism is enabled.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

3. Timing Functions

These functions measure elapsed time during program execution.

- **omp_get_wtime()**: Returns the elapsed wall-clock time (in seconds) since a fixed point in the past.
- **omp_get_wtick()**: Returns the precision of the timer used by omp_get_wtime().

4. Locking and Synchronization

These functions handle mutual exclusion and synchronization to avoid race conditions.

- **omp_init_lock(omp_lock_t *lock)**: Initializes a lock.
- **omp_set_lock(omp_lock_t *lock)**: Acquires a lock (blocks if the lock is already held).
- **omp_unset_lock(omp_lock_t *lock)**: Releases a lock.
- **omp_test_lock(omp_lock_t *lock)**: Attempts to acquire a lock without blocking (returns true if successful).
- **omp_destroy_lock(omp_lock_t *lock)**: Destroys a lock.

5. Environment Functions

These functions query or modify environment variables that affect OpenMP execution.

- **omp_get_dynamic()**: Returns whether dynamic adjustment of the number of threads is enabled.
- **omp_set_dynamic(int)**: Enables or disables dynamic adjustment of the number of threads.
- **omp_get_schedule()**: Returns the runtime schedule type and chunk size for loops.
- **omp_set_schedule(omp_sched_t, int chunk_size)**: Sets the schedule type and chunk size.

6. Task Management

These functions help create and manage tasks explicitly.

- **omp_get_task_team_size()**: Returns the size of the task team.
- **omp_get_task_num()**: Returns the ID of the current task.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Simple C program to Demonstrate OpenMP:

%%writefile OpenMP.c

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
 //Beginning of parallel region
 #pragma omp parallel
 {
   printf("Hello World...from thread = %d\n", omp_get_thread_num());
 }
 //End of parallel region
 }
```

Compilation and Execution:

```
[8]  %env OMP_NUM_THREADS=5
     env: OMP_NUM_THREADS=5

[9]  !gcc -o hello -fopenmp OpenMP.c

[11] !./hello
```

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8

Output:

Conclusion:

The demonstration of OpenMP highlights its powerful capabilities for implementing parallelism in shared-memory systems. By utilizing simple and flexible directives, runtime functions, and environment variables, OpenMP enables developers to optimize computational performance with minimal programming effort.

Name: SHAIK SHAWUL HAMID
Enrollment No: 2303031247050
Division: 6AI8